

LAPORAN PRATIKUM STRUKTUR DATA

“BINARY TREE DAN GRAPH”



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : Dr. WAHYUDI,
S.T, M.T

ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI
INFORMASI DEPARTEMEN
INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data Binary Tree dan Graph serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	3
2.1 Dasar Teori.....	3
2.2 Penjelasan Kode Program	4
2.2.1 Class Node_2511531007.....	4
2.2.2 Class BTree_2511531007	6
2.2.3 Class BTreeDriver_2511531007	8
2.2.4 Class GraphTraversal_2511531007.....	11
2.3 Tugas Praktikum	14
2.3.1 Class PetaKampusUnand_2511531007	14
BAB III PENUTUP	19
3.1 Kesimpulan	19
3.2 Saran	19
DAFTAR PUSTAKA.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk menyimpan dan mengelola data agar dapat diproses secara efisien. Selain struktur data linear, terdapat juga struktur data non-linear seperti Tree dan Graph yang digunakan untuk merepresentasikan hubungan antar data. Struktur data tersebut banyak diterapkan dalam berbagai bidang, seperti sistem pencarian, jaringan komputer, dan pengolahan data.

Pada praktikum ini digunakan struktur data Binary Tree dan Graph. Pada Binary Tree dilakukan operasi pembentukan tree, perhitungan jumlah node, serta traversal menggunakan metode Inorder, Preorder, dan Postorder. Sedangkan pada Graph dilakukan penelusuran simpul menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Seluruh implementasi dilakukan menggunakan bahasa pemrograman Java.

Melalui praktikum ini diharapkan dapat memahami konsep dan cara kerja struktur data Binary Tree dan Graph, serta mengetahui perbedaan metode traversal dan algoritma penelusuran yang digunakan dalam proses pencarian dan pengolahan data.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dan implementasi struktur data Binary Tree beserta metode traversal Inorder, Preorder, dan Postorder menggunakan bahasa pemrograman Java.
2. Memahami konsep dan implementasi struktur data Graph serta algoritma penelusuran Depth First Search (DFS) dan Breadth First Search (BFS) menggunakan bahasa pemrograman Java.

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Menambah pemahaman mengenai cara membangun dan menelusuri struktur data Binary Tree sehingga lebih mudah memahami hubungan antar node dalam sebuah tree.

2. Menambah pemahaman mengenai cara kerja algoritma DFS dan BFS pada Graph sehingga dapat mengetahui proses penelusuran dan pencarian simpul pada suatu graf.

BAB II PEMBAHASAN

2.1 Dasar Teori

Struktur data Tree merupakan salah satu struktur data non-linear yang digunakan untuk menyimpan data dalam bentuk hierarki. Tree terdiri dari sekumpulan node yang saling terhubung, di mana terdapat satu node utama yang disebut root serta node-node lain yang berperan sebagai parent dan child. Struktur Tree banyak digunakan dalam berbagai aplikasi, seperti sistem file, struktur organisasi, dan representasi data yang memiliki hubungan bertingkat.

Binary Tree adalah jenis Tree yang setiap node-nya memiliki paling banyak dua anak, yaitu anak kiri (left child) dan anak kanan (right child). Pada Binary Tree terdapat beberapa metode traversal yang digunakan untuk mengunjungi seluruh node dalam tree. Traversal Preorder dilakukan dengan urutan root, subtree kiri, kemudian subtree kanan. Traversal Inorder dilakukan dengan urutan subtree kiri, root, kemudian subtree kanan. Sedangkan traversal Postorder dilakukan dengan urutan subtree kiri, subtree kanan, kemudian root. Metode traversal ini digunakan untuk menampilkan atau memproses data yang tersimpan pada tree dengan urutan tertentu.

Graph merupakan struktur data non-linear yang terdiri dari kumpulan simpul (vertex) dan hubungan antar simpul yang disebut sisi (edge). Berbeda dengan Tree yang memiliki struktur hierarki, Graph digunakan untuk merepresentasikan hubungan yang lebih kompleks antar data. Graph banyak digunakan dalam berbagai bidang, seperti jaringan komputer, media sosial, sistem navigasi, dan pemetaan rute.

Depth First Search (DFS) merupakan algoritma penelusuran pada Graph yang bekerja dengan mengunjungi simpul sedalam mungkin sebelum kembali ke simpul sebelumnya. Algoritma ini umumnya menggunakan pendekatan rekursif atau struktur data stack untuk menyimpan simpul yang akan diproses. DFS sering digunakan untuk pencarian jalur, penelusuran struktur Graph, dan penyelesaian berbagai permasalahan yang melibatkan hubungan antar simpul.

Breadth First Search (BFS) adalah algoritma penelusuran pada Graph yang bekerja dengan mengunjungi seluruh simpul yang berada pada tingkat yang sama terlebih dahulu sebelum berpindah ke tingkat berikutnya. Algoritma ini menggunakan struktur data queue untuk mengatur urutan simpul yang akan dikunjungi. BFS banyak digunakan untuk mencari jalur terpendek pada Graph tak berbobot dan untuk menelusuri hubungan antar simpul secara bertahap.

2.2 Penjelasan Kode Program

2.2.1 Class Node_2511531007

```
1 package pekan9_2511531007;
2
3 public class Node_2511531007 {
4     int data_1007;
5     Node_2511531007 left_1007;
6     Node_2511531007 right_1007;
7
8     public Node_2511531007(int data_1007) {
9         this.data_1007 = data_1007;
10        left_1007 = null;
11        right_1007 = null;
12    }
13    public void setLeft_1007(Node_2511531007 node_1007) {
14        if (left_1007 == null)
15            left_1007 = node_1007;
16    }
17    public void setRight_1007(Node_2511531007 node_1007) {
18        if (right_1007 == null)
19            right_1007 = node_1007;
20    }
21    public Node_2511531007 getLeft_1007() {
22        return left_1007;
23    }
24    public Node_2511531007 getRight_1007() {
25        return right_1007;
26    }
27    public int getData_1007() {
28        return data_1007;
29    }
30    public void setData_1007(int data_1007) {
31        this.data_1007 = data_1007;
32    }
33
34    void printPreorder_1007(Node_2511531007 node_1007) {
35        if (node_1007 == null)
36            return;
37        System.out.print(node_1007.data_1007 + " ");
38        printPreorder_1007(node_1007.left_1007);
39        printPreorder_1007(node_1007.right_1007);
40    }
41    void printPostorder_1007(Node_2511531007 node_1007) {
42        if (node_1007 == null)
43            return;
44        printPostorder_1007(node_1007.left_1007);
45        printPostorder_1007(node_1007.right_1007);
46        System.out.print(node_1007.data_1007 + " ");
47    }
48    void printInorder_1007(Node_2511531007 node_1007) {
49        if (node_1007 == null)
50            return;
51        printInorder_1007(node_1007.left_1007);
52        System.out.print(node_1007.data_1007 + " ");
53        printInorder_1007(node_1007.right_1007);
54    }
55    public String print_1007() {
56        return this.print_1007("", true, "");
57    }
58    public String print_1007(String prefix_1007, boolean isTail_1007, String sb_1007) {
59        if (right_1007 != null) {
60            right_1007.print_1007(prefix_1007 + (isTail_1007 ? "| " : " "), false, sb_1007);
61        }
62        System.out.println(prefix_1007 + (isTail_1007 ? "\\-- " : "/-- ") + data_1007);
63        if (left_1007 != null) {
64            left_1007.print_1007(prefix_1007 + (isTail_1007 ? " " : "| "), true, sb_1007);
65        }
66        return sb_1007;
67    }
68 }
```

Kode program di atas digunakan untuk merepresentasikan sebuah node pada struktur data Binary Tree. Setiap node memiliki data yang disimpan dalam variabel `data_1007` serta dua buah referensi, yaitu `left_1007` dan `right_1007`, yang digunakan untuk menunjuk ke node anak kiri dan anak kanan. Class ini juga menyediakan method untuk mengatur, mengambil, dan menampilkan isi tree menggunakan beberapa jenis traversal.

Di dalam class `Node_2511531007` terdapat konstruktor `Node_2511531007(int data_1007)` yang berfungsi untuk membuat node baru dan menginisialisasi nilai data sesuai dengan parameter yang diberikan. Pada saat node dibuat, referensi `left_1007` dan

right_1007 diatur bernilai null yang menandakan bahwa node tersebut belum memiliki anak.

Method setLeft_1007() dan setRight_1007() digunakan untuk menambahkan node anak ke sisi kiri dan kanan. Kedua method ini hanya akan mengisi referensi anak jika posisi tersebut masih kosong (null). Selain itu, terdapat method getLeft_1007(), getRight_1007(), dan getData_1007() yang berfungsi untuk mengambil nilai anak kiri, anak kanan, dan data yang tersimpan pada node. Method setData_1007() digunakan untuk mengubah nilai data yang terdapat pada node.

Program ini juga menyediakan beberapa method traversal tree, yaitu printPreorder_1007(), printPostorder_1007(), dan printInorder_1007(). Pada method printPreorder_1007(), data node akan ditampilkan terlebih dahulu, kemudian dilanjutkan dengan traversal ke subtree kiri dan subtree kanan. Pada method printPostorder_1007(), traversal dilakukan ke subtree kiri dan subtree kanan terlebih dahulu, kemudian data node ditampilkan. Sedangkan pada method printInorder_1007(), traversal dilakukan ke subtree kiri, kemudian data node ditampilkan, lalu dilanjutkan ke subtree kanan.

Ketiga method traversal tersebut menggunakan teknik rekursif. Pada setiap method terdapat kondisi if (node_1007 == null) yang berfungsi sebagai kondisi penghentian rekursi. Jika node yang diproses bernilai null, maka method akan langsung berhenti dan kembali ke pemanggilan sebelumnya.

Selain traversal, terdapat method print_1007() yang digunakan untuk menampilkan struktur Binary Tree dalam bentuk visual menyerupai diagram pohon. Method ini memanggil method print_1007(String prefix_1007, boolean isTail_1007, String sb_1007) yang bekerja secara rekursif untuk mencetak node beserta hubungan antara parent dan child. Subtree kanan dicetak terlebih dahulu, kemudian node saat ini, dan terakhir subtree kiri. Dengan cara ini, struktur tree dapat ditampilkan secara vertikal sehingga lebih mudah dipahami.

2.2.2 Class BTree_2511531007

```
1 package pekan9_2511531007;
2
3 public class BTree_2511531007 {
4     private Node_2511531007 root_1007;
5     private Node_2511531007 currentNode_1007;
6     public BTree_2511531007() {
7         root_1007 = null;
8     }
9     public boolean search_1007(int data_1007) {
10         return search_1007(root_1007, data_1007);
11     }
12     private boolean search_1007(Node_2511531007 node_1007, int data_1007) {
13         if (node_1007.getData_1007() == data_1007)
14             return true;
15         if (node_1007.getLeft_1007() != null)
16             if (search_1007(node_1007.getLeft_1007(), data_1007))
17                 return true;
18         if (node_1007.getRight_1007() != null)
19             if (search_1007(node_1007.getRight_1007(), data_1007))
20                 return true;
21         return false;
22     }
23     public void printInorder_1007() {
24         root_1007.printInorder_1007(root_1007);
25     }
26     public void printPreorder_1007() {
27         root_1007.printPreorder_1007(root_1007);
28     }
29     public void printPostorder_1007() {
30         root_1007.printPostorder_1007(root_1007);
31     }
32     public Node_2511531007 getRoot_1007() {
33         return root_1007;
34     }
35 }
36
37 public boolean isEmpty_1007() {
38     return root_1007 == null;
39 }
40
41 public int countNodes_1007() {
42     return countNodes_1007(root_1007);
43 }
44 private int countNodes_1007(Node_2511531007 node_1007) {
45     int count_1007 = 1;
46     if (node_1007 == null) {
47         return 0;
48     } else {
49         count_1007 += countNodes_1007(node_1007.getLeft_1007());
50         count_1007 += countNodes_1007(node_1007.getRight_1007());
51         return count_1007;
52     }
53 }
54 public void print_1007() {
55     root_1007.print_1007();
56 }
57 public Node_2511531007 getCurrent_1007() {
58     return currentNode_1007;
59 }
60 public void setCurrent_1007(Node_2511531007 node_1007) {
61     this.currentNode_1007 = node_1007;
62 }
63 public void setRoot_1007(Node_2511531007 root_1007) {
64     this.root_1007 = root_1007;
65 }
```

Kode program di atas digunakan untuk merepresentasikan struktur data Binary Tree yang berfungsi untuk mengelola seluruh node yang terdapat di dalam tree. Class BTree_2511531007 memiliki variabel root_1007 yang digunakan untuk menyimpan node akar (root) dari tree serta variabel currentNode_1007 yang digunakan untuk menyimpan node yang sedang aktif atau sedang diproses. Class ini menyediakan method untuk melakukan pencarian data, menghitung jumlah node, menampilkan traversal, dan mencetak struktur tree.

Di dalam class BTree_2511531007 terdapat constructor BTree_2511531007() yang berfungsi untuk membuat tree baru. Pada saat tree dibuat, variabel root_1007

diinisialisasi dengan nilai null yang menandakan bahwa tree masih kosong dan belum memiliki node.

Method `search_1007(int data_1007)` digunakan untuk melakukan pencarian data pada tree. Method ini memanggil method rekursif `search_1007(Node_2511531007 node_1007, int data_1007)` yang bertugas menelusuri seluruh node dalam tree. Pertama, program akan membandingkan data pada node saat ini dengan data yang dicari. Jika data ditemukan, method akan mengembalikan nilai true. Jika tidak, program akan melanjutkan pencarian ke subtree kiri dan subtree kanan secara rekursif hingga data ditemukan atau seluruh node selesai diperiksa. Jika data tidak ditemukan, method akan mengembalikan nilai false.

Program ini juga menyediakan beberapa method traversal tree, yaitu `printInorder_1007()`, `printPreorder_1007()`, dan `printPostorder_1007()`. Ketiga method tersebut digunakan untuk menampilkan isi tree dengan urutan traversal yang berbeda. Method `printInorder_1007()` akan menampilkan data dengan urutan subtree kiri, root, kemudian subtree kanan. Method `printPreorder_1007()` akan menampilkan data dengan urutan root, subtree kiri, kemudian subtree kanan. Sedangkan method `printPostorder_1007()` akan menampilkan data dengan urutan subtree kiri, subtree kanan, kemudian root. Ketiga method ini memanggil method traversal yang telah dibuat pada class `Node_2511531007`.

Selain traversal, terdapat method `isEmpty_1007()` yang digunakan untuk memeriksa apakah tree masih kosong atau tidak. Method ini akan mengembalikan nilai true jika `root_1007` bernilai null dan false jika tree sudah memiliki node. Terdapat juga method `getRoot_1007()` yang digunakan untuk mengambil node root dari tree.

Program ini juga menyediakan method `countNodes_1007()` yang digunakan untuk menghitung jumlah seluruh node yang terdapat dalam tree. Method ini memanggil method rekursif `countNodes_1007(Node_2511531007 node_1007)`. Jika node yang diperiksa bernilai null, method akan mengembalikan nilai 0. Jika node tidak bernilai null, maka program akan menghitung satu node saat ini kemudian menambahkan jumlah node pada subtree kiri dan subtree kanan. Dengan cara ini, seluruh node dalam tree dapat dihitung secara rekursif.

Selain itu, terdapat method `print_1007()` yang digunakan untuk menampilkan struktur Binary Tree secara visual. Method ini memanggil method `print_1007()` yang terdapat pada class `Node_2511531007`, sehingga bentuk tree dapat ditampilkan

menyerupai diagram pohon dan memudahkan pengguna untuk melihat hubungan antara node induk dan node anak.

Terakhir, terdapat method `getCurrent_1007()`, `setCurrent_1007()`, dan `setRoot_1007()` yang berfungsi sebagai getter dan setter untuk mengakses serta mengubah nilai `currentNode_1007` dan `root_1007`. Method-method ini memudahkan proses pengelolaan node yang sedang aktif maupun node root pada Binary Tree.

2.2.3 Class BTreeDriver_2511531007

```
1 package pekan9_2511531007;
2
3 public class BTreeDriver_2511531007 {
4
5     public static void main(String[] args) {
6
7         //Membuat pohon
8         BTree_2511531007 tree_1007 = new BTree_2511531007();
9         System.out.println("Jumlah Simpul awal pohon: ");
10        System.out.println(tree_1007.countNodes_1007());
11
12        //Menambahkan simpul data 1
13        Node_2511531007 root_1007 = new Node_2511531007(1);
14
15        // Meniadikan simpul 1 sebagai root
16        tree_1007.setRoot_1007(root_1007);
17        System.out.println("Jumlah simpul jika hanya ada root");
18        System.out.println(tree_1007.countNodes_1007());
19        Node_2511531007 node2_1007 = new Node_2511531007(2);
20        Node_2511531007 node3_1007 = new Node_2511531007(3);
21        Node_2511531007 node4_1007 = new Node_2511531007(4);
22        Node_2511531007 node5_1007 = new Node_2511531007(5);
23        Node_2511531007 node6_1007 = new Node_2511531007(6);
24        Node_2511531007 node7_1007 = new Node_2511531007(7);
25        Node_2511531007 node8_1007 = new Node_2511531007(8);
26        Node_2511531007 node9_1007 = new Node_2511531007(9);
27        root_1007.setLeft_1007(node2_1007);
28        node2_1007.setLeft_1007(node4_1007);
29        node2_1007.setRight_1007(node5_1007);
30        node4_1007.setRight_1007(node8_1007);
31        root_1007.setRight_1007(node3_1007);
32        node3_1007.setLeft_1007(node6_1007);
33        node3_1007.setRight_1007(node7_1007);
34        node6_1007.setLeft_1007(node9_1007);
35
36        //Set root
37        tree_1007.setCurrent_1007(tree_1007.getRoot_1007());
38        System.out.println("menampilkan simpul terakhir: ");
39        System.out.println(tree_1007.getCurrent_1007().getData_1007());
40        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
41        System.out.println(tree_1007.countNodes_1007());
42        System.out.println("InOrder: ");
43        tree_1007.printInorder_1007();
44        System.out.println("\nPreorder: ");
45        tree_1007.printPreorder_1007();
46        System.out.println("\nPostorder: ");
47        tree_1007.printPostorder_1007();
48        System.out.println("\nmenampilkan simpul dalam bentuk pohon");
49        tree_1007.print_1007();
50    }
51 }
```

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk pohon
|
| /-- 7
| /-- 3
| | \-- 6
| | \-- 9
|-- 1
| /-- 5
|-- 2
| /-- 8
|-- 4

```

Kode program di atas digunakan untuk menguji implementasi struktur data Binary Tree yang telah dibuat pada class BTree_2511531007 dan Node_2511531007. Program ini berfungsi untuk membuat sebuah pohon biner, menambahkan beberapa simpul (node), menghitung jumlah simpul, menampilkan hasil traversal, serta mencetak struktur tree dalam bentuk visual.

Di dalam method main(), pertama-tama dibuat sebuah objek tree bernama tree_1007 menggunakan constructor BTree_2511531007(). Setelah itu program menampilkan jumlah simpul awal pada tree menggunakan method countNodes_1007(). Karena tree baru saja dibuat dan belum memiliki root, jumlah simpul yang ditampilkan adalah 0.

Selanjutnya program membuat sebuah node baru dengan nilai data 1 menggunakan constructor Node_2511531007(1). Node tersebut disimpan ke dalam variabel root_1007 dan kemudian dijadikan sebagai root tree menggunakan method setRoot_1007(root_1007). Setelah root ditambahkan, program kembali menghitung jumlah simpul pada tree. Pada tahap ini jumlah simpul yang dimiliki tree adalah 1 karena hanya terdapat node root.

Program kemudian membuat beberapa node tambahan dengan nilai 2, 3, 4, 5, 6, 7, 8, dan 9. Setiap node disimpan ke dalam variabel masing-masing, yaitu node2_1007, node3_1007, node4_1007, node5_1007, node6_1007, node7_1007, node8_1007, dan

node9_1007. Node-node tersebut kemudian dihubungkan menggunakan method `setLeft_1007()` dan `setRight_1007()` sehingga membentuk struktur Binary Tree.

Proses penghubungan node dilakukan dengan menjadikan node 2 sebagai anak kiri dari root dan node 3 sebagai anak kanan dari root. Selanjutnya node 4 dan node 5 menjadi anak kiri dan kanan dari node 2. Node 8 menjadi anak kanan dari node 4. Pada sisi kanan tree, node 6 dan node 7 menjadi anak kiri dan kanan dari node 3, sedangkan node 9 menjadi anak kiri dari node 6. Dengan hubungan tersebut terbentuk sebuah Binary Tree yang terdiri dari sembilan simpul.

Setelah seluruh node selesai ditambahkan, program mengatur `currentNode_1007` menggunakan method `setCurrent_1007()` dengan nilai root yang diperoleh dari method `getRoot_1007()`. Kemudian program menampilkan data pada node yang sedang aktif menggunakan method `getCurrent_1007().getData_1007()`. Karena current node diatur sama dengan root, maka data yang ditampilkan adalah nilai 1.

Selanjutnya program kembali menghitung jumlah simpul menggunakan method `countNodes_1007()`. Karena tree telah memiliki sembilan node, maka hasil yang ditampilkan adalah jumlah seluruh simpul yang terdapat pada tree tersebut.

Program juga menampilkan isi tree menggunakan tiga jenis traversal. Method `printInorder_1007()` digunakan untuk menampilkan data dengan urutan subtree kiri, root, kemudian subtree kanan. Method `printPreorder_1007()` digunakan untuk menampilkan data dengan urutan root, subtree kiri, kemudian subtree kanan. Sedangkan method `printPostorder_1007()` digunakan untuk menampilkan data dengan urutan subtree kiri, subtree kanan, kemudian root. Ketiga traversal tersebut membantu pengguna melihat perbedaan urutan kunjungan node pada Binary Tree.

Terakhir, program memanggil method `print_1007()` untuk menampilkan struktur Binary Tree dalam bentuk visual menyerupai diagram pohon. Dengan tampilan ini, hubungan antara node induk dan node anak dapat dilihat dengan lebih jelas sehingga memudahkan pengguna dalam memahami bentuk tree yang telah dibuat.

2.2.4 Class GraphTraversal_2511531007

```
1 package pekan9_2511531007;
2
3 import java.util.*;
4 public class GraphTraversal_2511531007 {
5     private Map<String, List<String>> graph_1007 = new HashMap<>();
6
7     //Menambahkan edge (graf tak berarah)
8     public void addEdge_1007(String node1_1007, String node2_1007) {
9         graph_1007.putIfAbsent(node1_1007, new ArrayList<>());
10        graph_1007.putIfAbsent(node2_1007, new ArrayList<>());
11        graph_1007.get(node1_1007).add(node2_1007);
12        graph_1007.get(node2_1007).add(node1_1007);
13    }
14    //Menampilkan graf awal
15    public void printGraph_1007() {
16        System.out.println("Graf Awal (Adjacency List):");
17        for (String node_1007 : graph_1007.keySet()) {
18            System.out.print(node_1007 + " -> ");
19            List<String> neighbors_1007 = graph_1007.get(node_1007);
20            System.out.println(String.join(", ", neighbors_1007));
21        }
22        System.out.println();
23    }
24    // DFS rekursif
25    public void dfs_1007(String start_1007) {
26        Set<String> visited_1007 = new HashSet<>();
27        System.out.println("Penelusuran DFS:");
28        dfsHelper_1007(start_1007, visited_1007);
29        System.out.println();
30    }
31    private void dfsHelper_1007(String current_1007, Set<String> visited_1007) {
32        if (visited_1007.contains(current_1007)) {
33            return;
34        }
35        visited_1007.add(current_1007);
36        System.out.print(current_1007 + " ");
37        for (String neighbor_1007 : graph_1007.getOrDefault(current_1007, new ArrayList<>())) {
38            dfsHelper_1007(neighbor_1007, visited_1007);
39        }
40    }
41    //BFS iteratif
42    public void bfs_1007(String start_1007) {
43        Set<String> visited_1007 = new HashSet<>();
44        Queue<String> queue_1007 = new LinkedList<>();
45        queue_1007.add(start_1007);
46        visited_1007.add(start_1007);
47        System.out.println("Penelusuran BFS:");
48        while (!queue_1007.isEmpty()) {
49            String current_1007 = queue_1007.poll();
50            System.out.print(current_1007 + " ");
51            for (String neighbor_1007 : graph_1007.getOrDefault(current_1007, new ArrayList<>())) {
52                if (!visited_1007.contains(neighbor_1007)) {
53                    queue_1007.add(neighbor_1007);
54                    visited_1007.add(neighbor_1007);
55                }
56            }
57            System.out.println();
58        }
59    }
60    //Main
61    public static void main(String[] args) {
62        GraphTraversal_2511531007 graph_1007 = new GraphTraversal_2511531007();
63
64        // Contoh graf: A-B, A-C, B-D, B-E
65        graph_1007.addEdge_1007("A", "B");
66        graph_1007.addEdge_1007("A", "C");
67        graph_1007.addEdge_1007("B", "D");
68        graph_1007.addEdge_1007("B", "E");
69
70        //Cetak graf awal
71        System.out.println("Graf Awal adalah: ");
72        graph_1007.printGraph_1007();
73
74        //Lakukan penelusuran
75        graph_1007.dfs_1007("A");
76        graph_1007.bfs_1007("A");
77    }
78 }
79 }
```

Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

Kode program di atas digunakan untuk merepresentasikan sebuah graf tak berarah serta melakukan proses penelusuran menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Graf disimpan dalam bentuk Adjacency List menggunakan struktur data `Map<String, List<String>>` yang memungkinkan setiap simpul menyimpan daftar simpul tetangganya.

Di dalam class `GraphTraversal_2511531007` terdapat variabel `graph_1007` yang bertipe `Map<String, List<String>>`. Variabel ini digunakan untuk menyimpan seluruh simpul beserta hubungan antar simpul dalam graf. Setiap key pada map merepresentasikan sebuah simpul, sedangkan value berupa list yang berisi simpul-simpul yang terhubung dengannya.

Method `addEdge_1007(String node1_1007, String node2_1007)` digunakan untuk menambahkan sisi atau hubungan antara dua simpul. Karena graf yang digunakan merupakan graf tak berarah, maka hubungan ditambahkan ke kedua arah. Pertama program memastikan bahwa kedua simpul sudah tersedia pada map menggunakan `putIfAbsent()`. Setelah itu, `node2_1007` ditambahkan ke daftar tetangga `node1_1007`, dan `node1_1007` juga ditambahkan ke daftar tetangga `node2_1007`.

Program juga menyediakan method `printGraph_1007()` yang digunakan untuk menampilkan isi graf dalam bentuk Adjacency List. Method ini melakukan perulangan pada seluruh simpul yang terdapat dalam map, kemudian menampilkan setiap simpul beserta daftar tetangganya. Dengan tampilan ini, hubungan antar simpul pada graf dapat dilihat dengan lebih mudah.

Untuk melakukan penelusuran menggunakan algoritma DFS, program menyediakan method `dfs_1007(String start_1007)`. Method ini membuat sebuah `HashSet` bernama `visited_1007` yang digunakan untuk menyimpan simpul-simpul yang sudah dikunjungi agar tidak diproses berulang kali. Selanjutnya method `dfsHelper_1007()` dipanggil untuk melakukan proses penelusuran secara rekursif mulai dari simpul awal yang diberikan.

Pada method `dfsHelper_1007(String current_1007, Set<String> visited_1007)`, program terlebih dahulu memeriksa apakah simpul saat ini sudah pernah dikunjungi. Jika sudah, method akan langsung berhenti menggunakan perintah `return`. Jika belum, simpul akan ditambahkan ke dalam `visited_1007` dan ditampilkan ke layar. Setelah itu program melakukan perulangan pada seluruh tetangga dari simpul tersebut dan

memanggil kembali method `dfsHelper_1007()` secara rekursif. Dengan cara ini, DFS akan menelusuri simpul sedalam mungkin sebelum kembali ke simpul sebelumnya.

Selain DFS, program juga menyediakan method `bfs_1007(String start_1007)` yang digunakan untuk melakukan penelusuran menggunakan algoritma Breadth First Search (BFS). Method ini menggunakan struktur data Queue bernama `queue_1007` untuk menyimpan simpul yang akan diproses. Simpul awal dimasukkan ke dalam queue dan ditandai sebagai sudah dikunjungi menggunakan `visited_1007`.

Proses BFS dilakukan menggunakan perulangan `while` selama queue masih berisi data. Pada setiap perulangan, simpul yang berada di depan queue diambil menggunakan method `poll()` dan ditampilkan ke layar. Selanjutnya program memeriksa seluruh tetangga dari simpul tersebut. Jika terdapat tetangga yang belum pernah dikunjungi, maka simpul tersebut akan dimasukkan ke dalam queue dan ditandai sebagai sudah dikunjungi. Dengan cara ini, BFS akan menelusuri graf berdasarkan tingkat kedekatan simpul, yaitu seluruh tetangga terdekat akan dikunjungi terlebih dahulu sebelum berpindah ke tingkat berikutnya.

Di dalam method `main()`, dibuat sebuah objek `GraphTraversal_2511531007` bernama `graph_1007`. Program kemudian membangun graf dengan menambahkan beberapa hubungan antar simpul, yaitu A-B, A-C, B-D, dan B-E menggunakan method `addEdge_1007()`. Hubungan-hubungan tersebut membentuk sebuah graf sederhana yang terdiri dari lima simpul.

Setelah graf selesai dibuat, program menampilkan isi graf menggunakan method `printGraph_1007()`. Selanjutnya dilakukan penelusuran menggunakan algoritma DFS dan BFS dengan titik awal simpul A melalui pemanggilan method `dfs_1007("A")` dan `bfs_1007("A")`. Hasil yang ditampilkan menunjukkan urutan kunjungan simpul berdasarkan metode DFS dan BFS sehingga pengguna dapat melihat perbedaan cara kerja kedua algoritma tersebut.

2.3 Tugas Praktikum

2.3.1 Class PetaKampusUnand_2511531007

```
1 package tugasPraktikum_pekan9_2511531007;
2
3 import java.awt.*;
4 import java.util.*;
5 import java.util.List;
6 import javax.swing.*;
7 import javax.swing.border.*;
8
9 public class PetaKampusUnand_2511531007 extends JFrame {
10
11     private static final long serialVersionUID = 1L;
12
13     static class GraphTraversal_1007 {
14
15         private Map<String, List<String>> graph_1007 = new LinkedHashMap<>();
16
17         public void addEdge_1007(String node1_1007, String node2_1007) {
18             graph_1007.putIfAbsent(node1_1007, new ArrayList<>());
19             graph_1007.putIfAbsent(node2_1007, new ArrayList<>());
20             graph_1007.get(node1_1007).add(node2_1007);
21             graph_1007.get(node2_1007).add(node1_1007);
22         }
23
24         public List<String[]> getAllEdges_1007() {
25             List<String[]> edges_1007 = new ArrayList<>();
26             Set<String> added_1007 = new HashSet<>();
27             for (String node_1007 : graph_1007.keySet()) {
28                 for (String neighbor_1007 : graph_1007.get(node_1007)) {
29                     String key1_1007 = node_1007 + "-" + neighbor_1007;
30                     String key2_1007 = neighbor_1007 + "-" + node_1007;
31                     if (!added_1007.contains(key1_1007) && !added_1007.contains(key2_1007)) {
32                         edges_1007.add(new String[] { node_1007, neighbor_1007 });
33                         added_1007.add(key1_1007);
34                     }
35                 }
36             }
37             return edges_1007;
38         }
39     }
40
41     // BFS Iteratif
42     public TraversalResult_1007 bfs_1007(String start_1007, String goal_1007) {
43         List<String> visited_1007 = new ArrayList<>();
44         Map<String, String> parent_1007 = new LinkedHashMap<>();
45         Queue<String> queue_1007 = new LinkedList<>();
46         queue_1007.add(start_1007);
47         visited_1007.add(start_1007);
48         parent_1007.put(start_1007, null);
49         while (!queue_1007.isEmpty()) {
50             String current_1007 = queue_1007.poll();
51             if (current_1007.equals(goal_1007)) {
52                 break;
53             }
54             for (String neighbor_1007 : graph_1007.getOrDefault(current_1007, new ArrayList<>())) {
55                 if (!visited_1007.contains(neighbor_1007)) {
56                     queue_1007.add(neighbor_1007);
57                     visited_1007.add(neighbor_1007);
58                     parent_1007.put(neighbor_1007, current_1007);
59                 }
60             }
61         }
62         return new TraversalResult_1007(visited_1007, buildPath_1007(parent_1007, start_1007, goal_1007));
63     }
64
65     // DFS Rekursif
66     public TraversalResult_1007 dfs_1007(String start_1007, String goal_1007) {
67         List<String> visited_1007 = new ArrayList<>();
68         Map<String, String> parent_1007 = new LinkedHashMap<>();
69         parent_1007.put(start_1007, null);
70         dfsHelper_1007(start_1007, goal_1007, visited_1007, parent_1007);
71         return new TraversalResult_1007(visited_1007, buildPath_1007(parent_1007, start_1007, goal_1007));
72     }
73
74     private boolean dfsHelper_1007(String current_1007, String goal_1007, List<String> visited_1007,
75                                   Map<String, String> parent_1007) {
76         if (visited_1007.contains(current_1007)) {
77             return false;
78         }
79         visited_1007.add(current_1007);
80         if (current_1007.equals(goal_1007)) {
81             return true;
82         }
83         for (String neighbor_1007 : graph_1007.getOrDefault(current_1007, new ArrayList<>())) {
84             if (!visited_1007.contains(neighbor_1007)) {
85                 parent_1007.put(neighbor_1007, current_1007);
86                 if (dfsHelper_1007(neighbor_1007, goal_1007, visited_1007, parent_1007)) {
87                     return true;
88                 }
89             }
90         }
91         return false;
92     }
93
94     private List<String> buildPath_1007(Map<String, String> parent_1007, String start_1007, String goal_1007) {
95         List<String> path_1007 = new ArrayList<>();
96         if (!parent_1007.containsKey(goal_1007)) {
97             return path_1007;
98         }
99         String cur_1007 = goal_1007;
100         while (cur_1007 != null) {
101             path_1007.add(0, cur_1007);
102             cur_1007 = parent_1007.get(cur_1007);
103         }
104         return path_1007;
105     }
106
107     static class TraversalResult_1007 {
108         public final List<String> visited_1007;
109         public final List<String> path_1007;
110
111         TraversalResult_1007(List<String> v, List<String> p) {
112             visited_1007 = v;
113             path_1007 = p;
114         }
115     }
116
117     class GraphPanel_1007 extends JPanel {
118
119         private static final long serialVersionUID = 2L;
120         private static final int R_1007 = 35;
121
122         @Override
123         protected void paintComponent(Graphics g_1007) {
124             super.paintComponent(g_1007);
125             Graphics2D g2_1007 = (Graphics2D) g_1007;
126             g2_1007.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
127             drawEdges_1007(g2_1007);
128             drawNodes_1007(g2_1007);
129             drawLegend_1007(g2_1007);
130         }
131     }
132 }
```

```

127 private void drawEdges_1007(Graphics2D g2_1007) {
128     for (String[] edge_1007 : traversal_1007.getAllEdges_1007()) {
129         int[] p1_1007 = nodePos_1007.get(edge_1007[0]);
130         int[] p2_1007 = nodePos_1007.get(edge_1007[1]);
131         if (p1_1007 == null || p2_1007 == null)
132             continue;
133         if (isEdgeOnPath_1007(edge_1007[0], edge_1007[1])) {
134             g2_1007.setColor(color.ORANGE);
135             g2_1007.setStroke(new BasicStroke(3f));
136         } else {
137             g2_1007.setColor(color.DARK_GRAY);
138             g2_1007.setStroke(new BasicStroke(1.5f));
139         }
140         g2_1007.drawLine(p1_1007[0], p1_1007[1], p2_1007[0], p2_1007[1]);
141     }
142     g2_1007.setStroke(new BasicStroke(1f));
143 }
144
145 private boolean isEdgeOnPath_1007(String a_1007, String b_1007) {
146     if (pathNodes_1007.size() < 2)
147         return false;
148     for (int i_1007 = 0; i_1007 < pathNodes_1007.size() - 1; i_1007++) {
149         String u_1007 = pathNodes_1007.get(i_1007);
150         String v_1007 = pathNodes_1007.get(i_1007 + 1);
151         if ((u_1007.equals(a_1007) && v_1007.equals(b_1007))
152             || (u_1007.equals(b_1007) && v_1007.equals(a_1007)))
153             return true;
154     }
155     return false;
156 }
157

```

```

158 private void drawNodes_1007(Graphics2D g2_1007) {
159     for (String node_1007 : nodePos_1007.keySet()) {
160         int[] pos_1007 = nodePos_1007.get(node_1007);
161         int x_1007 = pos_1007[0];
162         int y_1007 = pos_1007[1];
163
164         Color fill_1007;
165         if ((pathNodes_1007.isEmpty() && node_1007.equals(pathNodes_1007.get(0))) {
166             fill_1007 = new Color(40, 200, 113); // start
167         } else if ((pathNodes_1007.isEmpty()
168             && node_1007.equals(pathNodes_1007.get(pathNodes_1007.size() - 1))) {
169             fill_1007 = new Color(231, 76, 60); // goal
170         } else if (pathNodes_1007.contains(node_1007)) {
171             fill_1007 = new Color(241, 196, 15); // jalur
172         } else if (visitedNodes_1007.contains(node_1007)) {
173             fill_1007 = new Color(52, 122, 219); // dikunjungi
174         } else {
175             fill_1007 = new Color(189, 195, 199); // belum
176         }
177
178         g2_1007.setColor(fill_1007);
179         g2_1007.fillOval(x_1007 - R_1007, y_1007 - R_1007, R_1007 * 2, R_1007 * 2);
180         g2_1007.setColor(color.BLACK);
181         g2_1007.setStroke(new BasicStroke(1.5f));
182         g2_1007.drawOval(x_1007 - R_1007, y_1007 - R_1007, R_1007 * 2, R_1007 * 2);
183         g2_1007.setStroke(new BasicStroke(1f));
184
185         g2_1007.setFont(new Font("SansSerif", Font.BOLD, 10));
186         g2_1007.setColor(color.BLACK);
187         FontMetrics fm_1007 = g2_1007.getFontMetrics();
188         String[] parts_1007 = node_1007.split(" ", 2);
189         if (parts_1007.length == 2) {
190             g2_1007.drawString(parts_1007[0], x_1007 - fm_1007.stringWidth(parts_1007[0]) / 2, y_1007 - 3);
191             g2_1007.drawString(parts_1007[1], x_1007 - fm_1007.stringWidth(parts_1007[1]) / 2, y_1007 + 11);
192         } else {
193             g2_1007.drawString(node_1007, x_1007 - fm_1007.stringWidth(node_1007) / 2, y_1007 + 4);
194         }
195     }
196 }

```

```

197 private void drawLegend_1007(Graphics2D g2_1007) {
198     int lx_1007 = 40;
199     int ly_1007 = getHeight() - 105;
200     g2_1007.setFont(new Font("SansSerif", Font.PLAIN, 11));
201     String[][] items_1007 = { { "Lokasi awal", "" }, { "Lokasi Tujuan", "" }, { "Jalur Ditemukan", "" },
202         { "Sudah Dikunjungi", "" }, { "Belum Dikunjungi", "" } };
203     Color[] colors_1007 = { new Color(40, 200, 113), new Color(231, 76, 60), new Color(241, 196, 15),
204         new Color(52, 122, 219), new Color(189, 195, 199) };
205     for (int i_1007 = 0; i_1007 < items_1007.length; i_1007++) {
206         g2_1007.setColor(colors_1007[i_1007]);
207         g2_1007.fillRect(lx_1007, ly_1007, 14, 14);
208         g2_1007.setColor(color.BLACK);
209         g2_1007.drawRect(lx_1007, ly_1007, 14, 14);
210         g2_1007.drawString(items_1007[i_1007][0], lx_1007 + 20, ly_1007 + 12);
211         ly_1007 += 20;
212     }
213 }
214
215 private JComboBox<String> cbStart_1007;
216 private JComboBox<String> cbGoal_1007;
217 private JTextArea taResult_1007;
218 private JPanel panel_1007;
219
220 private final GraphTraversal_1007 traversal_1007 = new GraphTraversal_1007();
221 private List<String> visitedNodes_1007 = new ArrayList<>();
222 private List<String> pathNodes_1007 = new ArrayList<>();
223 private final Map<String, int> nodePos_1007 = new LinkedHashMap<>();
224
225 private static final String[] NODES_1007 = { "Gerbang Utama", "Rektorat", "Perpus", "FIT", "FEB", "FH", "FK", "FTI",
226     "Masjid Nurul Ilmi", "Auditorium" };
227
228 public static void main(String[] args) {
229     EventQueue.invokeLater() -> {
230         try {
231             new PetakKampusIlham_2511511007().setVisible(true);
232         } catch (Exception e) {
233             e.printStackTrace();
234         }
235     };
236 }
237

```

```

240 public PetaKampusUnand_2511531007() {
241     setTitle("Peta Universitas Andalas - BFS & DFS");
242     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
243     setBounds(100, 100, 1150, 680);
244     buildGraph_1007();
245     setupUI_1007();
246 }
247
248 private void buildGraph_1007() {
249     nodePos_1007.put("Gerbang Utama", new int[] { 100, 220 });
250     nodePos_1007.put("FK", new int[] { 280, 80 });
251     nodePos_1007.put("Masjid Nurul Ilmi", new int[] { 280, 190 });
252     nodePos_1007.put("FEB", new int[] { 280, 300 });
253     nodePos_1007.put("FH", new int[] { 520, 240 });
254     nodePos_1007.put("FTI", new int[] { 780, 140 });
255     nodePos_1007.put("Perpus", new int[] { 780, 300 });
256     nodePos_1007.put("Auditorium", new int[] { 1000, 80 });
257     nodePos_1007.put("Rektorat", new int[] { 1000, 190 });
258     nodePos_1007.put("FT", new int[] { 1000, 300 });
259
260     traversal_1007.addEdge_1007("Gerbang Utama", "FK");
261     traversal_1007.addEdge_1007("Gerbang Utama", "Masjid Nurul Ilmi");
262
263     traversal_1007.addEdge_1007("FK", "Masjid Nurul Ilmi");
264     traversal_1007.addEdge_1007("FK", "FH");
265
266     traversal_1007.addEdge_1007("Masjid Nurul Ilmi", "FEB");
267     traversal_1007.addEdge_1007("Masjid Nurul Ilmi", "FH");
268
269     traversal_1007.addEdge_1007("FEB", "Perpus");
270     traversal_1007.addEdge_1007("FEB", "FH");
271
272     traversal_1007.addEdge_1007("FH", "FTI");
273     traversal_1007.addEdge_1007("FH", "Rektorat");
274
275     traversal_1007.addEdge_1007("FTI", "Auditorium");
276     traversal_1007.addEdge_1007("FTI", "Perpus");
277
278     traversal_1007.addEdge_1007("Perpus", "FT");
279     traversal_1007.addEdge_1007("Perpus", "Rektorat");
280
281     traversal_1007.addEdge_1007("FT", "Rektorat");
282     traversal_1007.addEdge_1007("FTI", "Rektorat");
283 }
284
285 private void setupUI_1007() {
286     JPanel contentPane_1007 = new JPanel(new BorderLayout(5, 5));
287     contentPane_1007.setBorder(new EmptyBorder(8, 8, 8, 8));
288     setContentPane(contentPane_1007);
289
290     // Panel atas
291     JPanel topPanel_1007 = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
292     topPanel_1007.setBorder(new TitledBorder("Pencarian Jalur Menggunakan BFS dan DFS"));
293
294     topPanel_1007.add(new JLabel("Lokasi Awal :"));
295     cbStart_1007 = new JComboBox<>(NODES_1007);
296     topPanel_1007.add(cbStart_1007);
297
298     topPanel_1007.add(new JLabel("Lokasi Tujuan :"));
299     cbGoal_1007 = new JComboBox<>(NODES_1007);
300     cbGoal_1007.setSelectedIndex(5);
301     topPanel_1007.add(cbGoal_1007);
302
303     JButton btnBFS_1007 = new JButton("BFS");
304     JButton btnDFS_1007 = new JButton("DFS");
305     JButton btnReset_1007 = new JButton("Reset");
306
307     btnBFS_1007.addActionListener(e -> runBFS_1007());
308     btnDFS_1007.addActionListener(e -> runDFS_1007());
309     btnReset_1007.addActionListener(e -> resetGraph_1007());
310
311     topPanel_1007.add(btnBFS_1007);
312     topPanel_1007.add(btnDFS_1007);
313     topPanel_1007.add(btnReset_1007);
314     contentPane_1007.add(topPanel_1007, BorderLayout.NORTH);
315
316     // Panel tengah
317     graphPanel_1007 = new GraphPanel_1007();
318     graphPanel_1007.setBorder(new TitledBorder("Visualisasi Graph"));
319     contentPane_1007.add(graphPanel_1007, BorderLayout.CENTER);
320
321     // Panel bawah
322     JPanel bottomPanel_1007 = new JPanel(new BorderLayout());
323     bottomPanel_1007.setBorder(new TitledBorder("Hasil Pencarian"));
324     bottomPanel_1007.setPreferredSize(new Dimension(4, 195));
325
326     taResult_1007 = new JTextArea();
327     taResult_1007.setEditable(false);
328     taResult_1007.setFont(new Font("Monospaced", Font.PLAIN, 12));
329     bottomPanel_1007.add(new JScrollPane(taResult_1007), BorderLayout.CENTER);
330     contentPane_1007.add(bottomPanel_1007, BorderLayout.SOUTH);
331 }
332
333

```

```

333 private void runBFS_1007() {
334     String start_1007 = (String) cbStart_1007.getSelectedItem();
335     String goal_1007 = (String) cbGoal_1007.getSelectedItem();
336     if (start_1007.equals(goal_1007)) {
337         showSameNodeWarning_1007();
338         return;
339     }
340     GraphTraversal_1007.TraversalResult_1007 res_1007 = traversal_1007.bfs_1007(start_1007, goal_1007);
341     visitedNodes_1007 = res_1007.visited_1007;
342     pathNodes_1007 = res_1007.path_1007;
343     displayResult_1007("BFS", start_1007, goal_1007, res_1007.visited_1007, res_1007.path_1007);
344     graphPanel_1007.repaint();
345 }
346
347 private void runDFS_1007() {
348     String start_1007 = (String) cbStart_1007.getSelectedItem();
349     String goal_1007 = (String) cbGoal_1007.getSelectedItem();
350     if (start_1007.equals(goal_1007)) {
351         showSameNodeWarning_1007();
352         return;
353     }
354     GraphTraversal_1007.TraversalResult_1007 res_1007 = traversal_1007.dfs_1007(start_1007, goal_1007);
355     visitedNodes_1007 = res_1007.visited_1007;
356     pathNodes_1007 = res_1007.path_1007;
357     displayResult_1007("DFS", start_1007, goal_1007, res_1007.visited_1007, res_1007.path_1007);
358     graphPanel_1007.repaint();
359 }
360
361 private void resetGraph_1007() {
362     visitedNodes_1007.clear();
363     pathNodes_1007.clear();
364     taResult_1007.setText("");
365     cbStart_1007.setSelectedIndex(0);
366     cbGoal_1007.setSelectedIndex(0);
367     graphPanel_1007.repaint();
368 }
369
370 private void displayResult_1007(String algo_1007, String start_1007, String goal_1007, List<String> visited_1007,
371     List<String> path_1007) {
372     StringBuilder sb_1007 = new StringBuilder();
373     sb_1007.append("Hasil Pencarian ").append(algo_1007).append("\n");
374     sb_1007.append("Lokasi Awal ").append(start_1007).append("\n");
375     sb_1007.append("Lokasi Tujuan ").append(goal_1007).append("\n");
376     sb_1007.append(".....\n");
377     sb_1007.append("Node dikunjungi ").append(String.join(" > ", visited_1007)).append("\n");
378     sb_1007.append("Jalur dieksplorasi ").append(visited_1007.size()).append(" node\n");
379     sb_1007.append(".....\n");
380     if (path_1007.isEmpty()) {
381         sb_1007.append("Jalur : Tidak ditemukan\n");
382     } else {
383         sb_1007.append("Jalur : ").append(String.join(" > ", path_1007)).append("\n");
384         sb_1007.append("Panjang Jalur : ").append(path_1007.size() - 1).append(" langkah\n");
385     }
386     taResult_1007.setText(sb_1007.toString());
387 }
388
389 private void showSameNodeWarning_1007() {
390     JOptionPane.showMessageDialog(this, "Lokasi awal dan tujuan tidak boleh sama!", "Peringatan",
391         JOptionPane.WARNING_MESSAGE);
392 }
393 }

```

Kode program di atas digunakan untuk membuat aplikasi pencarian jalur pada peta kampus Universitas Andalas menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Program dibuat menggunakan bahasa pemrograman Java dengan tampilan GUI (Graphical User Interface) menggunakan Java Swing. Selain menampilkan hasil pencarian jalur, program juga dapat memvisualisasikan graph kampus serta menunjukkan node yang telah dikunjungi selama proses pencarian.

Di dalam class GraphTraversal_1007 terdapat struktur data graph yang disimpan menggunakan Map<String, List<String>>. Method addEdge_1007() digunakan untuk menambahkan hubungan antar lokasi pada graph. Setiap hubungan bersifat dua arah sehingga lokasi yang terhubung dapat diakses dari kedua sisi. Method getAllEdges_1007() digunakan untuk mengambil seluruh edge yang terdapat pada graph agar dapat ditampilkan pada visualisasi.

Proses pencarian jalur dilakukan menggunakan algoritma BFS dan DFS. Method bfs_1007() melakukan penelusuran secara melebar menggunakan struktur data Queue. Algoritma ini akan mengunjungi seluruh node yang berada pada tingkat yang sama terlebih dahulu sehingga biasanya menghasilkan jalur terpendek. Sedangkan method dfs_1007() melakukan penelusuran secara mendalam menggunakan teknik rekursif. Algoritma ini akan menelusuri satu cabang hingga mencapai tujuan atau node terakhir sebelum kembali ke cabang lainnya.

Untuk menyimpan hasil pencarian digunakan class `TraversalResult_1007` yang berisi daftar node yang dikunjungi (`visited_1007`) dan jalur yang ditemukan (`path_1007`). Jalur tersebut dibentuk menggunakan method `buildPath_1007()` dengan memanfaatkan data parent dari setiap node yang telah dikunjungi.

Tampilan visual graph dibuat pada class `GraphPanel_1007` yang merupakan turunan dari `JPanel`. Method `paintComponent()` digunakan untuk menggambar node, edge, serta legenda warna. Method `drawEdges_1007()` digunakan untuk menggambar hubungan antar lokasi, sedangkan `drawNodes_1007()` digunakan untuk menggambar node dengan warna yang berbeda sesuai statusnya. Warna hijau menunjukkan lokasi awal, merah menunjukkan lokasi tujuan, kuning menunjukkan jalur yang ditemukan, biru menunjukkan node yang telah dikunjungi, dan abu-abu menunjukkan node yang belum dikunjungi.

Pada method `buildGraph_1007()`, program membentuk graph kampus Universitas Andalas yang terdiri dari beberapa lokasi seperti Gerbang Utama, Rektorat, Perpustakaan, Fakultas Teknik, Fakultas Ekonomi dan Bisnis, Fakultas Hukum, Fakultas Kedokteran, Fakultas Teknologi Informasi, Masjid Nurul Ilmi, dan Auditorium. Setiap lokasi dihubungkan menggunakan edge sesuai jalur yang telah ditentukan.

Pada method `setupUI_1007()`, dibuat tampilan GUI yang terdiri dari `ComboBox` untuk memilih lokasi awal dan tujuan, tombol BFS, DFS, dan Reset, panel visualisasi graph, serta area untuk menampilkan hasil pencarian. Ketika tombol BFS atau DFS ditekan, program akan menjalankan proses pencarian jalur, menampilkan node yang dikunjungi, jumlah node yang dieksplorasi, serta jalur yang ditemukan. Tombol Reset digunakan untuk mengembalikan tampilan ke kondisi awal.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Binary Tree dan Graph dapat digunakan untuk menyimpan serta mengelola data yang memiliki hubungan tertentu. Pada Binary Tree, data dapat ditelusuri menggunakan metode Inorder, Preorder, dan Postorder, sedangkan pada Graph proses penelusuran dapat dilakukan menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Meskipun metode penelusuran yang digunakan berbeda, seluruh node atau simpul tetap dapat dikunjungi dan diproses sesuai aturan masing-masing algoritma.

Melalui praktikum ini, dapat dipahami bagaimana proses pembentukan Binary Tree dan Graph serta cara kerja berbagai metode traversal dan algoritma penelusuran yang digunakan. Selain itu, praktikum ini juga membantu dalam memahami implementasi struktur data non-linear menggunakan bahasa pemrograman Java serta mengetahui perbedaan cara kerja antara traversal pada Binary Tree dengan algoritma DFS dan BFS pada Graph.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Dengan adanya latihan yang cukup, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

- [1] W. Wahyudi, *Struktur Data dengan Java*. CV Eureka Media Aksara, 2025. [Online].
Available: <https://books.google.co.id/books?id=EN21EQAAQBAJ>.